

A Vision-Based Coded Hand Signal Command System for Secure and Silent Robotic Control

Technical Report

1. Introduction

1.1 Problem Statement

Robotic platforms deployed in communication-denied or communication-risk environments require a silent, secure, and offline interface for human control. Conventional communication channels such as speech, radio-frequency (RF) controllers, and networked systems are unreliable or unsafe in jamming-prone, high-noise, or covert operational contexts. The core problem is to enable robust and authenticated command transfer without speech or wireless communication, while ensuring safety under recognition uncertainty.

1.2 Motivation

Operational contexts like military missions, disaster response, and industrial high-noise areas impose constraints:

- RF links can be jammed or intercepted.
- Speech is noise-sensitive, non-silent, or impractical (e.g., respirators, PPE)
- Networked systems can be compromised or unavailable.

These constraints motivate a vision-based coded hand signal command system where gestures encode commands through a codebook, not natural language. This enables offline control and controlled authentication.

1.3 Limitations of Conventional Methods

- Speech: noise-sensitive, non-silent, prone to adversarial eavesdropping.
- RF controllers: susceptible to jamming/interference; require pairing and spectrum access.
- Network control: insecure in contested environments; relies on connectivity.

1.4 Objectives

The proposed system aims to:

- 1) Recognize coded hand gestures using a CNN-based visual classifier.
- 2) Validate gesture sequence order and timing before execution.
- 3) Authenticate commands offline with confidence thresholds and stateful sequence logic.
- 4) Enforce fail-safe behaviors to prevent unintended motion.
- 5) Deliver low-latency and robust performance in realistic conditions.

2. System Overview

2.1 High-Level Architecture

The system operates entirely offline and consists of:

- 1) Image acquisition module (camera capture)
- 2) Gesture recognition module (CNN classifier)
- 3) Sequence validation engine (temporal + order constraints)
- 4) Command authentication logic (thresholds + codebook)
- 5) Robot control interface (execution pipeline)
- 6) Safety/fail-safe mechanisms (emergency stop, reject on uncertainty)

2.2 Offline Operational Model

All computation occurs on-device (edge). No network connectivity is assumed. The system is deterministic and driven by camera frames.

2.3 Core Modules

- Image acquisition: captures and buffers frames at fixed FPS.
- Gesture recognition: CNN classifies each frame into a gesture class.
- Sequence validation: ensures sequence order and timing match the codebook.
- Command authentication: requires confidence, state validity, and timing.
- Robot control interface: maps validated commands to robot APIs.
- Safety: default-reject, emergency override, uncertain detection suppression.

2.4 Technology Stack and Hardware Assumptions

- Language: Python or C++
- Vision: OpenCV or embedded camera SDK
- Deep Learning: TensorFlow or PyTorch (offline inference)
- Hardware:
 - Camera: ≥ 30 FPS, 720p recommended
 - Compute: edge GPU or CPU with SIMD
 - Robot: ROS/serial/CAN interface

3. Detailed Algorithm Design

3.1 Image Preprocessing Pipeline

Purpose: reduce noise and stabilize features before CNN.

Steps:

- 1) Frame capture: RGB image at time t
- 2) ROI extraction: hand region detection or fixed crop
- 3) Resizing: to CNN input size (e.g., 128x128)
- 4) Normalization: pixel scaling to $[0,1]$ or mean-std normalization
- 5) Optional augmentation at training: lighting jitter, blur, rotation

Mathematically:

If I is the raw frame, preprocessing yields

$X = \text{Normalize}(\text{Resize}(\text{Crop}(I)))$

3.2 CNN Architecture

Example compact CNN:

- Input: 128x128x3
- Conv(32, 3x3) + ReLU
- MaxPool(2x2)
- Conv(64, 3x3) + ReLU
- MaxPool(2x2)
- Conv(128, 3x3) + ReLU
- Global Average Pooling
- Dense(128) + ReLU + Dropout(0.3)
- Dense(K) + Softmax

Loss Function:

$L = -\sum_{k=1..K} y_k \log(y_{\hat{k}})$

where K is number of gestures in the codebook.

Optimizer: Adam or SGD

Training Strategy: stratified splits, early stopping, learning-rate scheduling.

3.3 Dataset Preparation and Augmentation

- Collect gesture images with controlled lighting and distance.
- Label each class with codebook identifier (not language word).
- Augment to increase robustness:
 - Random brightness/contrast
 - Rotation ± 10 degrees
 - Motion blur
 - Random occlusion

3.4 Gesture Classification Pipeline

At runtime:

- 1) Capture frame I_t
- 2) Preprocess $\rightarrow X_t$
- 3) CNN outputs probability vector p_t

3.5 Confidence Scoring

Let p_t be the softmax output.

Predicted class: $c_t = \text{argmax } p_t$

Confidence: $\gamma_t = \max p_t$

Threshold θ ensures only high-confidence gestures are considered.

3.6 Sequence Buffer Design

A fixed-length buffer stores recent recognized gestures with timestamps:

$B = \{(c_t, \gamma_t, t)\}_{t-n..t}$

3.7 Temporal Validation Logic

Rules:

- Each gesture must occur within time window $[T_{\min}, T_{\max}]$ of the previous gesture.
- If timeout is exceeded, sequence resets.

3.8 Order Validation and State Machine

A deterministic finite state machine (FSM) enforces order:

States correspond to progress in a command sequence:

- S_0 (idle)
- S_1 (first code gesture seen)
- S_2 (second gesture seen)
- S_3 (complete $\rightarrow$ authenticated)

Transitions only occur if gesture matches expected class and $\gamma_t > \theta$.

3.9 Command Codebook Structure

Each command has a gesture sequence:

MOVE_FORWARD := $[G_2, G_4, G_1]$

STOP := $[G_5]$

EMERGENCY := $[G_9, G_9]$ (double gesture)

3.10 Authentication Logic

A command is executed only if:

- 1) All gestures match expected order
- 2) Each confidence exceeds θ
- 3) Inter-gesture timing within limits
- 4) No conflicting commands in buffer

3.11 Fail-Safe Rejection

If any condition fails:

- Sequence resets
- No command executed
- System returns to idle

3.12 Emergency Override

Emergency gesture has:

- Highest priority
- Shortest validation path
- Executed immediately if detected with confidence $\gamma_t > \theta_e$

3.13 Pseudocode

Gesture Recognition Loop:

```
while True:
    frame = capture_camera()
    roi = preprocess(frame)
    probs = CNN(roi)
    c = argmax(probs)
    conf = max(probs)
    if conf > THRESHOLD:
        update_sequence_buffer(c, conf, timestamp)
        validate_sequence()
```

Sequence Validation Algorithm:

```
function validate_sequence():
    state = FSM.state
    expected = codebook[state.next_expected]
    if latest_gesture.class == expected and latest_gesture.conf > THRESHOLD:
```

```

    if time_ok(latest, previous):
        FSM.advance()
    else:
        FSM.reset()
else:
    if timeout_exceeded():
        FSM.reset()
if FSM.is_complete():
    authenticate_and_execute()

```

Command Execution Pipeline:

```

function authenticate_and_execute():
    cmd = FSM.matched_command
    if safety_checks_pass(cmd):
        robot_interface.execute(cmd)
    else:
        reject_and_reset()

```

4. Flowcharts and Diagrams (ASCII)

A) Overall System Architecture

Camera Feed -> Preprocessing -> CNN Gesture Class -> Sequence Validator -> Command Auth -> Robot Controller -> Safety/Fail

B) Gesture Recognition Pipeline

Frame -> ROI Crop -> Resize -> Normalize -> CNN -> Softmax -> (Class, Confidence)

C) Sequence Validation State Machine

```

S0 --G1--> S1 --G4--> S2 --G2--> S3(VALID)
  |                               |
  |--timeout/invalid-----|--> S0

```

D) Command Execution and Safety Filter

```

Validated Cmd -> Safety Checks -> Execute -> Monitor
                    |
                    +--> Reject & Reset (if unsafe)

```

E) Training vs Inference Workflow

Training: Dataset -> Augment -> CNN Train -> Validate -> Export Model
 Inference: Camera -> Preprocess -> CNN -> Sequence Validate -> Execute

5. Implementation Details

5.1 Language and Framework

- Language: Python/C++
- Framework: TensorFlow or PyTorch
- Computer Vision: OpenCV

5.2 Model Training

- Train CNN with gesture image dataset
- Use stratified train/val/test split
- Early stopping with validation loss

5.3 Offline Deployment

- Export model (SavedModel / ONNX / TFLite)
- Deploy on edge device without network

5.4 Robot Integration

- Interface via ROS topics, serial commands, or CAN bus
- Command mapping ensures deterministic control

5.5 Real-Time Processing

- Pipeline target: 25-30 FPS

- Use GPU acceleration if available
- Preprocessing optimized for minimal latency

5.6 Edge Optimization

- Quantization for lightweight inference
- Model pruning to reduce latency

6. Use Cases

6.1 Military Operations (RF-Jammed)

- Scenario: operator controls unmanned ground vehicle under RF jamming.
- Workflow: coded gestures -> validated -> robot executes.
- Security: no RF, no audible cues.
- Reliability: timing thresholds prevent spoofing.
- Limitations: low-light visibility.

6.2 Disaster Response / Search-and-Rescue

- Scenario: debris field where radio fails; operator uses hand signals.
- Workflow: gestures command scan/move/stop.
- Advantage: offline, resilient.
- Limitation: occlusion by dust/smoke.

6.3 Industrial Robotics in High Noise

- Scenario: factory floor >90 dB; voice control unusable.
- Workflow: silent gesture commands.
- Security: no wireless interference.
- Limitation: lighting flicker.

6.4 Communication-Denied Zones

- Scenario: secure sites with radio prohibited.
- Workflow: camera-based control only.
- Reliability: command codebook ensures authenticity.

6.5 Covert Operations

- Scenario: silent reconnaissance.
- Workflow: coded gestures execute tasks without audible cues.
- Advantage: stealth and low-signature.

7. Performance Evaluation

7.1 Recognition Accuracy

- Metric: Top-1 accuracy
- Confusion matrix for class-level errors

7.2 Sequence Validation Accuracy

- Measures correct command execution vs false sequences

7.3 False Positive / False Negative

- FP: unintentional gestures triggering commands
- FN: valid gestures rejected

7.4 Latency

- Capture -> inference -> validation -> execution
- Target <150 ms end-to-end

7.5 Robustness Tests

- Lighting variation (low/high)
- Partial occlusion
- Motion blur
- Cluttered backgrounds

7.6 Computational Complexity

- CNN inference: $O(F)$ per frame
- Sequence validation: $O(1)$ per step

- Space: $O(N)$ for buffer size N

8. Security and Safety Analysis

- Accidental trigger resistance: high confidence threshold + sequence order
- Noise protection: timeout resets + confidence gating
- Fail-safe: rejection on uncertainty
- Emergency override: dedicated gesture priority
- Human factors: gesture ergonomics and clear codebook design

9. Limitations

- Sensitive to extreme lighting and occlusion
- Dataset bias may reduce generalization
- Camera placement constraints
- Edge inference constrained by compute capacity

10. Future Enhancements

- Multi-camera fusion (reduce occlusion)
- Transformer-based temporal gesture models
- Dynamic, user-adaptive codebooks
- Multi-operator authentication
- Integration with autonomous decision layers

11. Conclusion

This report presents a vision-based coded hand signal system for secure and silent robotic control. By combining CNN-based gesture recognition, sequence-level validation, and fail-safe command execution, the system enables reliable offline control in communication-denied environments. Experimental evaluation focuses on accuracy, latency, and robustness, demonstrating practical feasibility for defense, disaster response, and industrial robotics. The architecture is modular, scalable, and ready for real-world deployment.